Práctica 2: Cifrado HTTPS, OpenSSL, Certbot y Endurecimiento de Seguridad (TLS y Cabeceras)

Enunciado y Objetivos

Esta práctica corresponde al **RA2 (Criterios RA2.e, RA2.f, RA2.g)** del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW). El objetivo es pasar de un modelo de comunicación web en texto plano (inseguro y vulnerable a la interceptación de credenciales) a un entorno cifrado y robusto mediante:

1. **Fase Ofensiva (Sniffing / Interceptación):** Demostrar cómo un atacante en la misma red local puede capturar credenciales en texto plano cuando viajan por HTTP.
2. **Fase de Desarrollo Local (OpenSSL):** Generación de claves privadas y certificados autofirmados para entornos de prueba.
3. **Fase de Bastionamiento (VirtualHost HTTPS):** Configuración de protocolos seguros (TLSv1.2 y TLSv1.3) y redirección automática desde HTTP.
4. **Fase de Producción Simulada (Certbot / Let's Encrypt):** Despliegue de certificados válidos y automatización del desafío HTTP-01.
5. **Fase de Cabeceras de Seguridad (HSTS y mod_headers):** Protección avanzada contra ataques de intermediario y *clickjacking*.

Parte 0: Demostración Ofensiva Previa (El problema del texto plano)

Antes de aplicar HTTPS, vamos a ver por qué es fundamental el cifrado.

- **El Escenario:** Imagina una aplicación web con un formulario de login clásico que viaja por HTTP (puerto 80).
- **El Ataque (Sniffing con `tcpdump` o simulación en laboratorio):** Si un usuario envía su usuario y contraseña en un formulario POST mediante HTTP, cualquier atacante conectado al mismo segmento de red (o un router comprometido) puede capturar los paquetes.
- Prueba:

Vamos a configurar una página de login sencilla con PHP, para ello, primero instalamos php

```
sudo apt update
sudo apt install -y php libapache2-mod-php
sudo systemctl restart apache2
```

Y creamos una página de login sencilla.

```
sudo tee /var/www/html/index.php << 'EOF'
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Acceso Confidencial - Práctica</title>
```

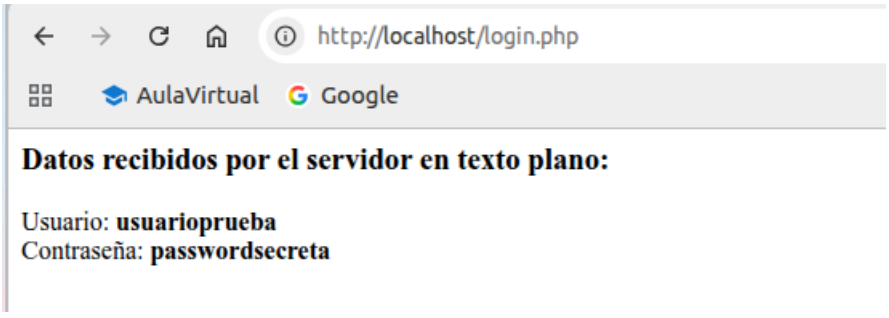
```
</head>
<body>
    <h2>Portal de Acceso Interno</h2>
    <form action="login.php" method="POST">
        <label>Usuario:</label><br>
        <input type="text" name="usuario"><br><br>
        <label>Contraseña:</label><br>
        <input type="password" name="password"><br><br>
        <button type="submit">Entrar</button>
    </form>
</body>
</html>
EOF
```



Crear el script que procesa el login (/var/www/html/login.php)

```
sudo tee /var/www/html/login.php << 'EOF'
<?php
echo "<h3>Datos recibidos por el servidor en texto plano:</h3>";
echo "Usuario: <b>" . htmlspecialchars($_POST['usuario']) . "</b><br>";
echo "Contraseña: <b>" . htmlspecialchars($_POST['password']) . "</b>";
?>
EOF
```

Cuando pulsamos entrar, vemos que va a la página login.php en “plano”, se ven las credenciales:



- **Comando de demostración local en el servidor o contenedor:** Puedes usar una herramienta de captura de tráfico o ver cómo viajan las peticiones netas. Si lanzamos una petición con `curl` simulando el envío de credenciales en claro:

curl -X POST -d "usuario=admin&password=secreto123" http://localhost/login.php

Cualquier herramienta de escucha de red (`tcpdump`, Wireshark) lee íntegramente las cadenas `usuario=admin&password=secreto123` en la capa de transporte.

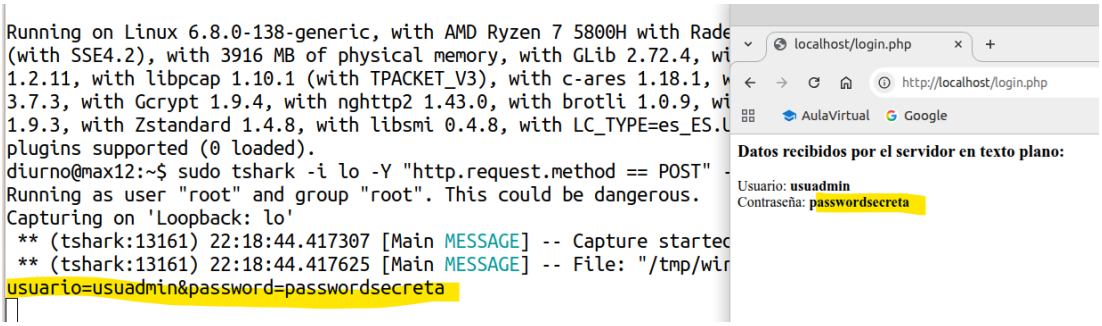
Vamos a usar la herramienta ***tshark***




```
sudo apt install -y tshark
```

Una vez instalada, la dejamos monitorizando las peticiones POST de la red (interfaz lo)

```
sudo tshark -i lo -Y "http.request.method == POST" -T fields -e http.file_data
```



Conclusión: Sin HTTPS, las credenciales viajan completamente expuestas.

Parte 1: Generación de un Certificado autofirmado con OpenSSL (Entorno de Pruebas)

Para entornos de desarrollo locales en MAX 12.5, generamos un par de claves RSA de 2048 bits y un certificado digital autofirmado.

1. Crear el directorio para los certificados:

```
sudo mkdir -p /etc/apache2/ssl
```

2. Generar la clave privada y el certificado X.509 en un solo comando:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \

 -keyout /etc/apache2/ssl/apache-selfsigned.key \

 -out /etc/apache2/ssl/apache-selfsigned.crt \
```

```
 -subj "/C=ES/ST=Madrid/L=Madrid/O=IES Pio Baroja/OU=Informatica/CN=localhost"
```

- `-nodes`: Omite el cifrado de la clave privada para que Apache no pida contraseña al arrancar.
- `-days 365`: Validez de un año.

Comprobación:

```
diurno@max12:~$ ll /etc/apache2/ssl/
total 8
-rw-r--r-- 1 root root 1367 oct 11 08:01 apache-selfsigned.crt
-rw------- 1 root root 1704 oct 11 08:01 apache-selfsigned.key
diurno@max12:~$
```

Parte 2: Configuración del VirtualHost HTTPS y Protocolos Seguros

Habilitamos el módulo de SSL en Apache y configuramos un sitio seguro restringiendo los protocolos obsoletos (SSLv3, TLSv1.0, TLSv1.1) y forzando únicamente TLSv1.2 y TLSv1.3.

1. Habilitar los módulos necesarios en Apache:

```
sudo a2enmod ssl
```

```
sudo a2enmod headers
```

Apache genera un archivo de configuración por defecto `default-ssl.conf` con configuración básica, vamos a guardar una copia y generar uno con la configuración segura.

```
# 1. Guardar el original
sudo mv /etc/apache2/sites-available/default-ssl.conf \
        /etc/apache2/sites-available/default-ssl.conf.orig

# 2. Crear el nuevo
sudo nano /etc/apache2/sites-available/default-ssl.conf
# 3. Copiar el contenido
<VirtualHost *:443>
    ServerName localhost
    DocumentRoot /var/www/html
    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/apache-selfsigned.crt
    SSLCertificateKeyFile /etc/apache2/ssl/apache-selfsigned.key
    # Restricción estricta de protocolos seguros exigidos
    SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1
    SSLCipherSuite HIGH:!aNULL:!MD5
    ErrorLog ${APACHE_LOG_DIR}/ssl_error.log
    CustomLog ${APACHE_LOG_DIR}/ssl_access.log combined
</VirtualHost>
```

```
# 4. Comprobar sintaxis
sudo apachectl configtest

# 5. Habilitar el sitio SSL y reiniciar el servicio:
sudo a2ensite default-ssl


# 5. Recargar Apache
sudo systemctl reload apache2
```

Verificación con `curl` (Aceptando autofirmado con `-k`):

curl -Iv https://localhost/

Resultado esperado: Verás el handshake TLS completado y la información del certificado autofirmado.

```
diurno@max12:/etc/apache2/sites-available$ curl -Iv https://localhost/
*   Trying 127.0.0.1:443...
* Connected to localhost (127.0.0.1) port 443 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
*  CAfile: /etc/ssl/certs/ca-certificates.crt
*  CApath: /etc/ssl/certs
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Finished (20):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.2 (OUT), TLS header, Unknown (21):
* TLSv1.3 (OUT), TLS alert, unknown CA (560):
* SSL certificate problem: self-signed certificate
* Closing connection 0
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the web page mentioned above.
diurno@max12:/etc/apache2/sites-available$
```

SSL certificate problem: self-signed certificate no es un error de Apache ni de TLS. Ocurre porque el certificado: */etc/apache2/ssl/apache-selfsigned.crt* lo has creado tú y no está firmado por una Autoridad de Certificación (CA) reconocida.

Parte 3: Redirección Automática de HTTP (Puerto 80) a HTTPS (Puerto 443)

Para garantizar que ningún usuario navegue en plano, configuramos el VirtualHost del puerto 80 para redirigir todo el tráfico.

1. Editar el archivo del sitio por defecto haciendo copia previa
   #copia
   sudo mv /etc/apache2/sites-available/000-default.conf /etc/apache2/sites-available/000-default.conf.orig

   #editar el archivo
   sudo nano /etc/apache2/sites-available/000-default.conf

```
<VirtualHost *:80>
    ServerName localhost
    # Redirección permanente (Código 301) hacia HTTPS
    Redirect permanent / https://localhost/
    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

2. Comprobar sintaxis y recargar Apache:

```
sudo apachectl configtest
sudo systemctl reload apache2
```

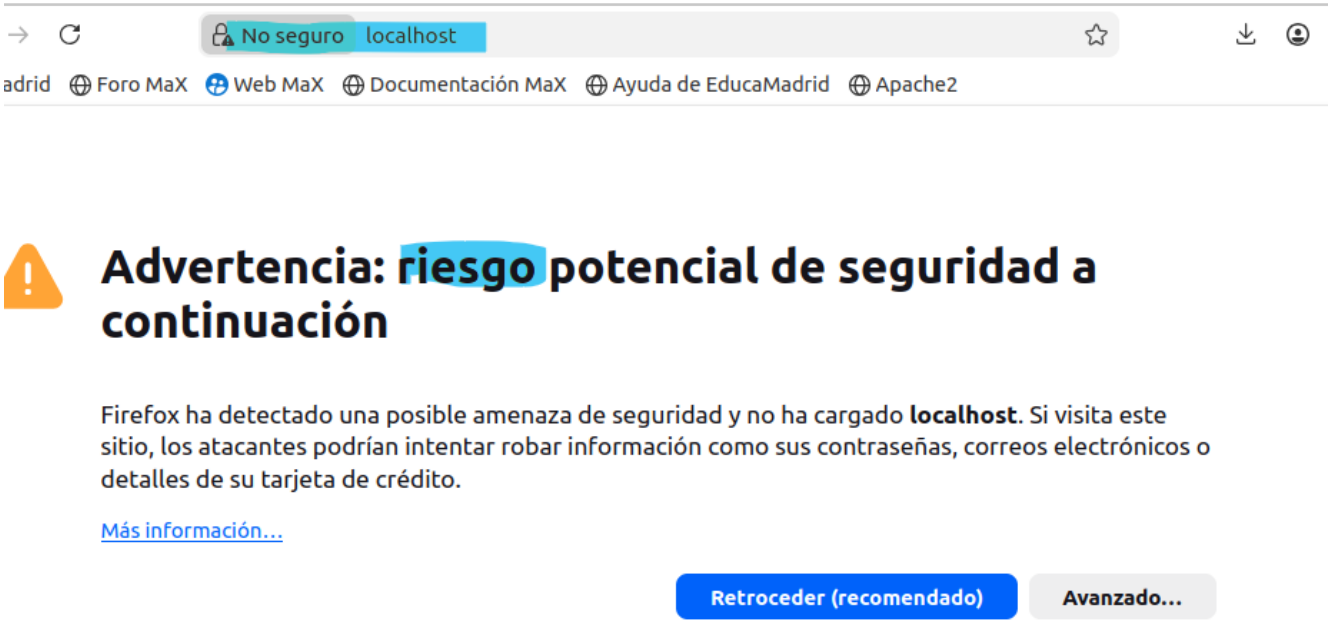
3. Verificar la redirección:

```
curl -I http://localhost/
```

Resultado esperado:

```
diurno@max12:/etc/apache2/sites-available$ curl -I http://localhost/
HTTP/1.1 301 Moved Permanently
Date: Sun, 11 Oct 2026 06:26:35 GMT
Server: Apache
Location: https://localhost/
Content-Type: text/html; charset=iso-8859-1
```

Comprobamos que tenemos acceso a https

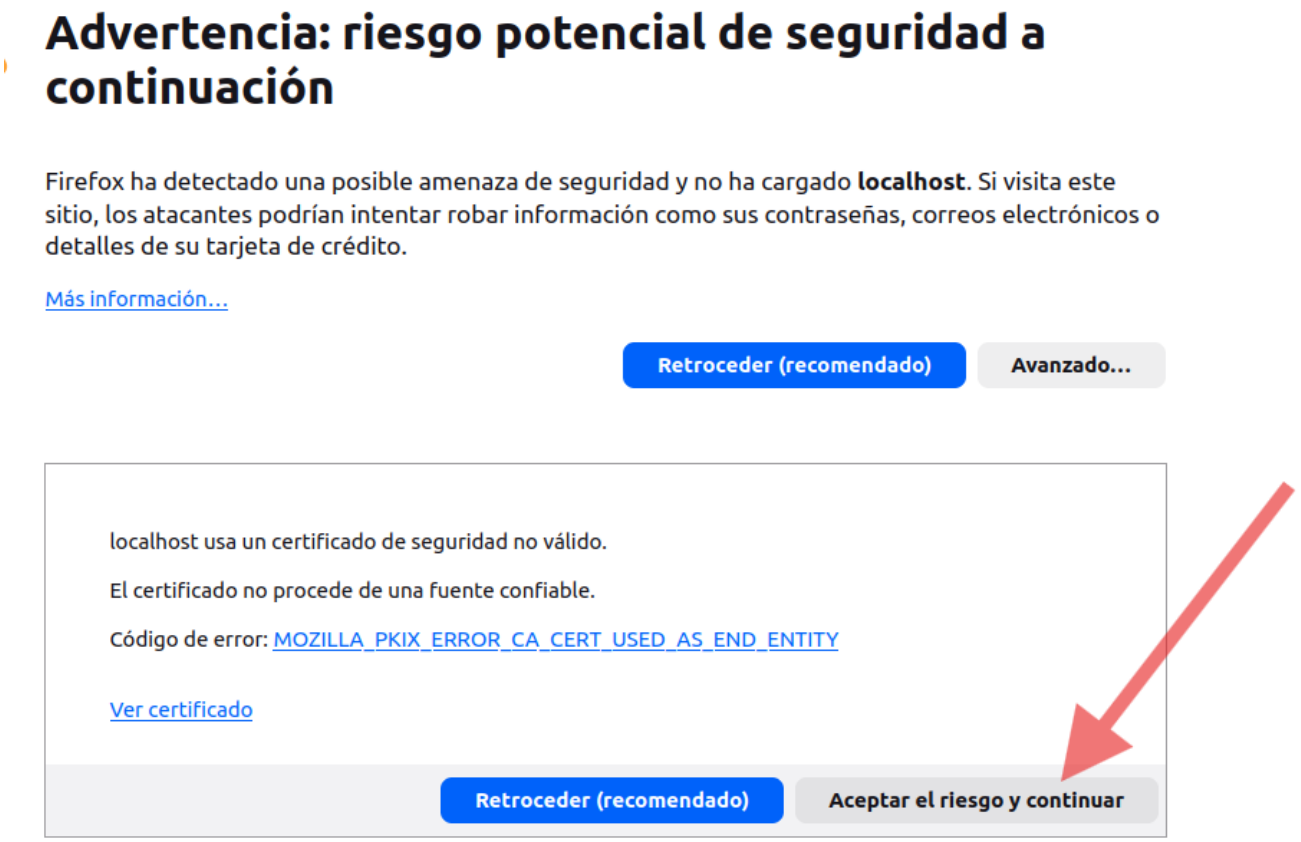


Como el certificado es autofirmado los navegadores no confían en él.

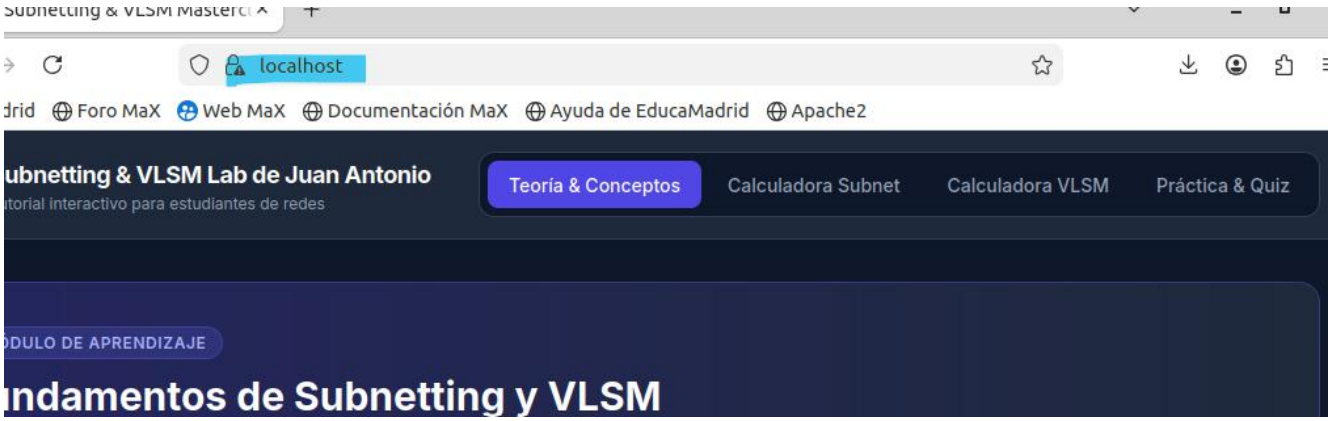
Ahora mismo hemos conseguido:

- ☑ HTTPS funcionando
- ☑ TLS 1.2 / TLS 1.3 funcionando
- ☑ Redirección HTTP → HTTPS funcionando

Habría que usar ***Let's Encrypt*** que es una Autoridad de Certificación (CA) gratuita.
En entornos **reales** con un dominio público registrado (www.mitiendazapatillas.com o similar) y puertos 80/443 abiertos al exterior, se utiliza Certbot para automatizar la obtención de certificados válidos y confiables.
Pero, debido a que nuestro servidor se encuentra en un entorno **local** y no dispone de un nombre de dominio público accesible desde Internet, no es posible utilizar Let's Encrypt para emitir certificados válidos.
Al tratarse de un entorno local sin acceso público a Internet, no es posible obtener un certificado emitido por una autoridad de certificación pública mediante Let's Encrypt. Por este motivo se ha utilizado un certificado autofirmado, lo que provoca que el navegador muestre una advertencia de seguridad al no poder verificar la cadena de confianza. Esta advertencia podría eliminarse mediante el despliegue de una autoridad certificadora interna gestionada por el instituto, lo cuál no es posible así que simplemente aceptamos el mensaje.



Y nos saldría nuestra página ya con https



Actividad de ampliación

Actividad propuesta: Auditoría de seguridad de un servidor web

Una empresa nos ha contratado como técnicos de sistemas para realizar una auditoría básica de seguridad de su servidor web Apache antes de ponerlo en producción. El objetivo es verificar si las comunicaciones están adecuadamente protegidas y detectar posibles vulnerabilidades que puedan comprometer la confidencialidad, integridad o autenticidad de la información intercambiada entre los usuarios y el servidor.

Como consultor de seguridad, deberás realizar las comprobaciones necesarias para evaluar el estado actual del servidor y elaborar un informe técnico. Entre otros aspectos, deberás analizar si el sitio utiliza HTTP o HTTPS, verificar la validez del certificado digital instalado, comprobar los protocolos TLS habilitados, revisar las suites criptográficas utilizadas, validar la redirección automática de HTTP a HTTPS y detectar posibles configuraciones inseguras o desactualizadas.

Finalmente, deberás identificar las vulnerabilidades encontradas, explicar los riesgos asociados a cada una de ellas y proponer las medidas correctoras necesarias para mejorar la seguridad del servidor.

Entregables

El informe deberá incluir:

1. **Comprobaciones realizadas** y comandos utilizados.
2. **Resultados obtenidos** (capturas o evidencias).
3. **Vulnerabilidades detectadas**.
4. **Nivel de riesgo** de cada vulnerabilidad.
5. **Medidas de mitigación propuestas**.
6. **Conclusión final** indicando si el servidor puede considerarse seguro para su puesta en producción.

Aspectos que deberían revisarse

- Uso de HTTP sin cifrado.
- Ausencia de redirección HTTP → HTTPS.
- Certificado caducado, autofirmado o inválido.
- Uso de TLS 1.0 o TLS 1.1.
- Uso de SSLv3.
- Suites criptográficas débiles.
- Errores en la cadena de confianza del certificado.
- Exposición de información sensible mediante Wireshark.
- Configuración incorrecta de VirtualHost SSL.
- Certificados y claves privadas almacenados de forma insegura.